

Exceções + Serialização



ESTGA

universidade de aveiro

PA – 18241

Programação Aplicada

Ricardo Marau

marau@ua.pt

Nesta aula

- Exceções
- Ficheiros
 - Leitura e escrita
- Linguagens de serialização de dados

```
if (file exists):  
    open file  
else:  
    ...
```

def open:

: Quando se faz o raise (o código é suspenso naquele sítio/momento), temos que tratar e lidar com essa exceção, e a função mas retorna nada

Exceções

- Evento que resulta de uma **quebra (erro) do fluxo normal de execução**.
 - Normalmente resulta de uma **anomalia**.
- Quando o Python não consegue lidar com uma situação e o **Código não pode prosseguir** levanta-se (**raise**) uma exceção.
 - Ex:

```
>>> 3/0
```

```
Traceback (most recent call last):  
  File "<stdin>", line 1, in <module>  
ZeroDivisionError: division by zero
```

Lidar com a Excepção

- O código que se pensa onde poderá ocorrer uma exceção coloca-se dentro de um bloco try/except.

try:

x=3/0

except ZeroDivisionError:

print("Excepção detetada.")

Opcional:

else: (não houve exceção)

print("Executou sem problema.")

Opcional:

finally: (acontece quer haja ou não exceção)

print("E finalmente... **Código de Limpeza**")

Modo de "guarda":
Try/Except
A escuta de determinada(s) exceção/exceções.

- Mais tipos de exceções:

<https://docs.python.org/3/library/exceptions.html>

try:

x=3/0

x=int("letras")

except ZeroDivisionError:
print("ZeroDivisionError detetada.")

except ValueError:
print("ValueError detetada.")

except:
print("Detetada outra exceção.")

(=) default das exceptions é Exception
Mas derivam todas de BaseException
(que em princípio não vamos
precisar de usar)

try:

x=3/0

x=int("letras")

except ZeroDivisionError as erro:
print("Detetada.", erro)

except ValueError as erro:
print("Detetada.", erro)

except:
print("Detetada outra exceção.")

raise something

↳ Objeto de um
Tipo de Dados (Classe)

try:

x=3/0

x=int("letras")

except Exception as erro:
print("Detetada.", erro)

if erro isinstance(...):

elif ...:

: KeyboardInterrupt
(Ctrl + C, Ctrl + D,
Ctrl + Z)

Mais um exemplo:

```
def getNumero():  
    while True:  
        try:  
            x = int(input("Insira um número:"))  
        except ValueError:  
            print("Tente novamente...")  
        else: break → facultativo  
        return x
```

except Exception:
pass

lidar c/a exceção, ignorando-a
o problema fica silencioso,
o problema
ainda existe, está
lá por resolver.
Adiar o problema

Forçar/Levantando uma exceção

- As exceções são "capturadas" no except, mas são geradas nalgum sítio.
- O `raise` permite gerar essas exceções em código.

```
def funcao( valor ):
```

```
    if valor < 1:
```

```
        raise Exception("Valor inserido inválido! " + str(valor))
```

o programa quebra/interrumpe (n é preciso else)

```
    print("Este código já não é executado se valor < 1.")
```

```
funcao(-1)
```

```
try:
```

```
    x = funcao(-1)
```

```
except:
```

```
    x = funcao(10)
```

Ficheiros I/O

```
f = open('ficheiro.txt', 'r')  
read_data = f.read()  
f.close()
```

Exepção:
FileNotFoundError

```
with open('ficheiro.txt') as f:  
    read_data = f.read()
```

```
if f.closed:  
    print("Ficheiro está fechado")
```

o with pode ser usado c/o with

MAS nem todos os objetos podem ser usados com o with (String)

só os que implementam `-- enter --` e `-- exit --` (Exception)

→ não lida com nenhuma exceção, mas não se esquece de fechar o ficheiro
(propaga para cima)

Abre e fecha o ficheiro no final
incluindo se houver Exceções

Quando um programa (main) crasha, todos os ficheiros fecham

Open modes

- 'r' default – Abre para leitura.
- 'w' Abre para escrita. Se não existe tenta criar. Se existir escreve por cima.
- 'x' Cria novo ficheiro. Aborta se ficheiro já existir.
- 'a' Como 'w' mas com cursor no fim – append.
- 't' Modo de texto.
- 'b' Modo binário. *rb/rb/ab/wb*

Ler e escrever

`f.read()` *#Lê o ficheiro todo*

`f.readline()` *#Lê uma linha e avança para a seguinte*

`f.write("Texto para o ficheiro")`

`f.write("Linha1\nLinha2")`

readlines → lê todas as linhas e armazena-as numa lista (não para ficheiros muito grandes)

Armazenar/Comunicar dados

- Formatos de ficheiros para guardar conteúdos (human-readable).

Multi-plataforma e Multi-Linguagem

- CSV (Comma-Separated Values)

- import **csv** *→ nativo em python*

- XML (Extensible Markup Language (XML)) *↗ estrutura parecida ao HTML*

- from **lxml** import etree

instalar lxml → é muito pesado

- JSON (JavaScript Object Notation)

- import **json**

- ... **YAML**

*↗ linguagem interpretada
↗ linguagem usada nos browsers*

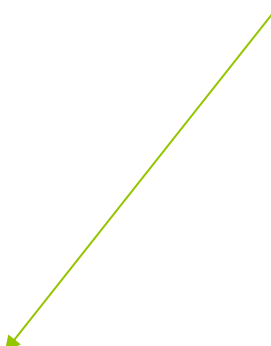
*{ tipo um print no python
Listas, dicionários, String(' ')
true / false, não há tuples, int e float*

CSV

'fx.csv'

Coluna1	Coluna2	Coluna3
Text1 na coluna1	texto2	texto3

```
import csv
with open('fx.csv', newline='') as csvfile:
    spamreader = csv.reader(csvfile, delimiter=' ', quotechar='|')
    for row in spamreader:
        print (' <-> '.join(row))
```



JSON - JavaScript Object Notation

'fx.json'

```
{"nome": "Manuel", "idade": 48, "isMan": true}
```

Está armazenado como um **dicionário** (chave-valor). Mas **podia ser uma lista de dicionários.**

import json

```
with open('fx.json') as f:  
    data = json.load(f)  
    print (data['nome'])
```

*string-ificar um objeto complexo
objeto JSON em Pyt*

```
data['idade'] = 20
```

```
with open('fx_2.json', 'w') as outfile:  
    json.dump(data, outfile)
```

*↳ voltar a escrever o data num
novo ficheiro json.*

*Tuples não existem
em JSON*

Serializar

Serialização { Transformar objeto
num formato representável
e enviá-lo pela Rede

- Imagine que pretende enviar/armazenar o conteúdo de um objeto.
De forma a poder reconstruir o objeto.

- Transforma o objeto numa String (pode ser binária). E vice-versa.

- Python

- marshal** – mais rápido

- pickle - completo

→ cria um ficheiro a partir de um Objeto
↳ só funciona de Python para Python (e com a mesma versão do pickle)

Serializar – a ter em atenção

- ◉ Quando o conteúdo serializado é armazenado ou transferido
 - ◉ Versões da classe (do objeto) devem ser idênticas
 - ◉ Versões do serializador (ex: pickle)
 - ◉ Só funciona de Python para Python
- ◉ Alternativas?
 - ◉ XML – JSON - etc

Serializar - Exemplo

```
class Fruta:
    def __init__(self, cor, valor):
        self.cor = cor
        self.valor = valor

banana = Fruta( 'amarelo', 30 )

import pickle

with open("Frutas.obj", "wb") as f:
    pickle.dump(banana, f)

with open("Frutas.obj", 'rb') as f:
    banana_nova = pickle.load(f)

print(banana_nova.cor, banana_nova.valor)
```


Fim

- Questões?